

EXP63 · GS_FLOATERLAB · 방향 설정 브리핑

예산 경쟁 구조와 다음 4방향

tracking · PGBA · map() · background_polish가 하나의 GPU를 어떻게 나눠 쓰는가
— 왜 같은 파라미터가 어떤 장면은 살리고 어떤 장면은 죽이는가

지금까지 봐온 패턴 — 결과만 알고 원인은 몰랐다

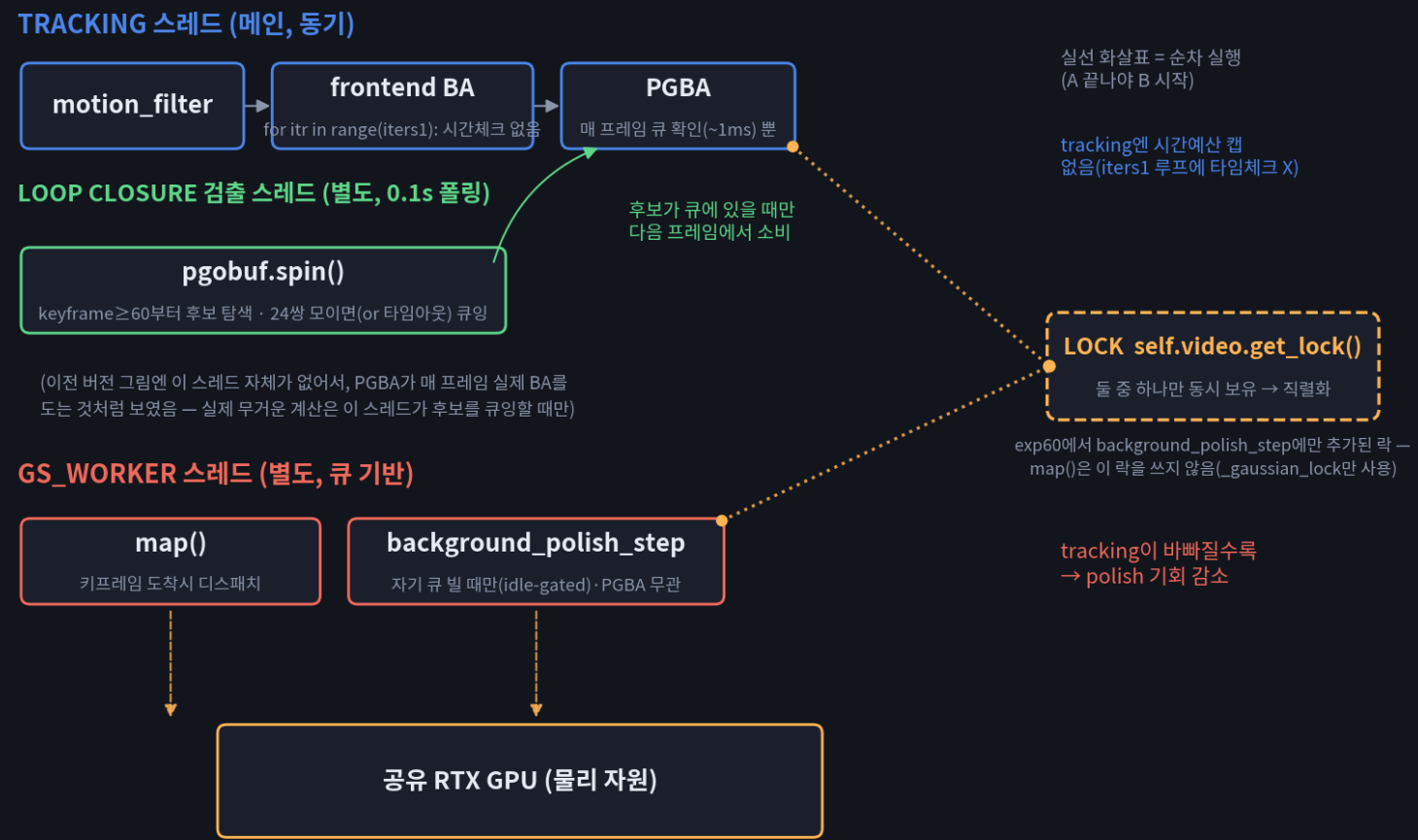
지금까지 방식

파라미터를 바꾸고 → dB가 오르내리는 걸 보고 → "이게 좋은가 보다"로 채택/기각. 축A2(thresh 2.6·iters1=2)가 정확히 이렇게 채택됐다 — 305에서 +7.0dB 보고 좋다고 판단, 근데 12F는 한 번도 검증 안 하고 방치했다가 나중에 -4.8dB 회귀를 발견했다.

이번에 요구되는 방식

각 함수(motion_filter/frontend/PGBA/map()/background_polish)가 실제로 몇 번 도는지, 1회당 몇 ms인지, 그 시간이 어떤 파라미터에 어떻게 스케일되는지를 먼저 재고, 그 지식으로 "왜 이 장면은 오르고 저 장면은 내렸는지"를 설명할 수 있어야 다음 파라미터를 결정한다.

세 스레드가 하나의 GPU를 나눠 쓰는 구조

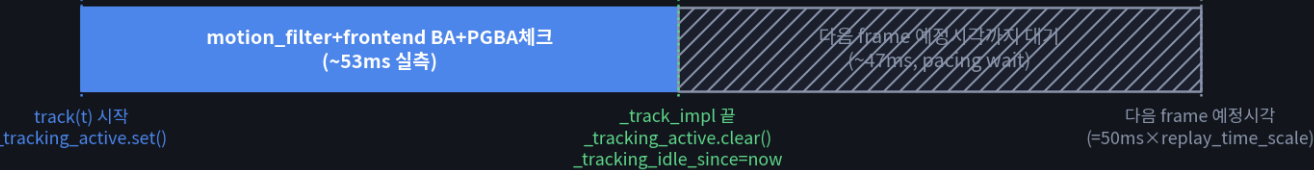


tracking 스레드는 동기·고정 반복(iters1)으로 돈다 — 시간 예산 캡이 코드 어디에도 없다. PGBA는 매 프레임 값싼 큐 확인만 하고, 실제 pose-graph BA는 별도 loop-closure 검출 스레드가 후보를 큐잉했을 때만 실행된다(상시 실행 아님). get_lock() 직렬화는 PGBA↔background_polish_step에만 적용되며 map()은 이 락을 쓰지 않는다.

프레임 1개 주기 안에서 무슨 일이 일어나는가

프레임 1개 처리 주기 — 코드 레벨 이벤트 (unit=10ms, scale=2.0 실측 기준)

TRACKING 스레드



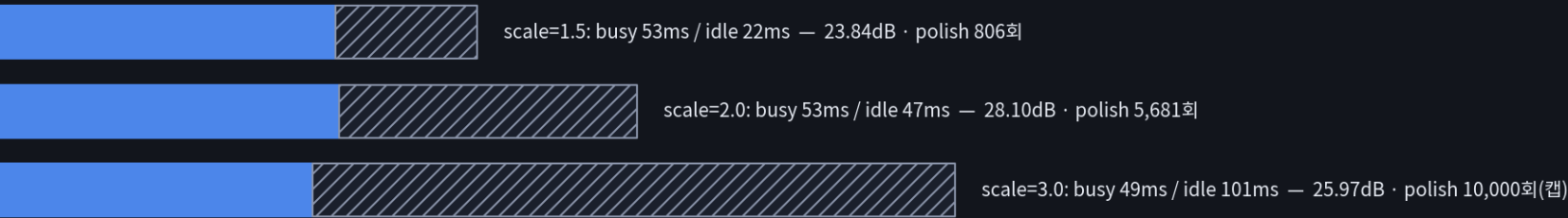
GS_WORKER 스레드 (poll, 0.2~2ms 주기)



idle 47ms 안에 background_polish_step(~4.9ms) 9회 낚

- 매 poll마다 4개 조건 AND 확인:
- ① self._background_polish
 - ② idle_long_enough (tracking 지금 idle?)
 - ③ under_budget (max_steps 안 넘음?)
 - ④ map_mature_enough (frame≥polish 시작점?)
- 전부 True일 때만 1회 실행 후 즉시 재확인

scale이 커지면 idle 구간(=폴리시 슬롯)이 늘어난다 (실측, 상대폭 75:100:150ms)

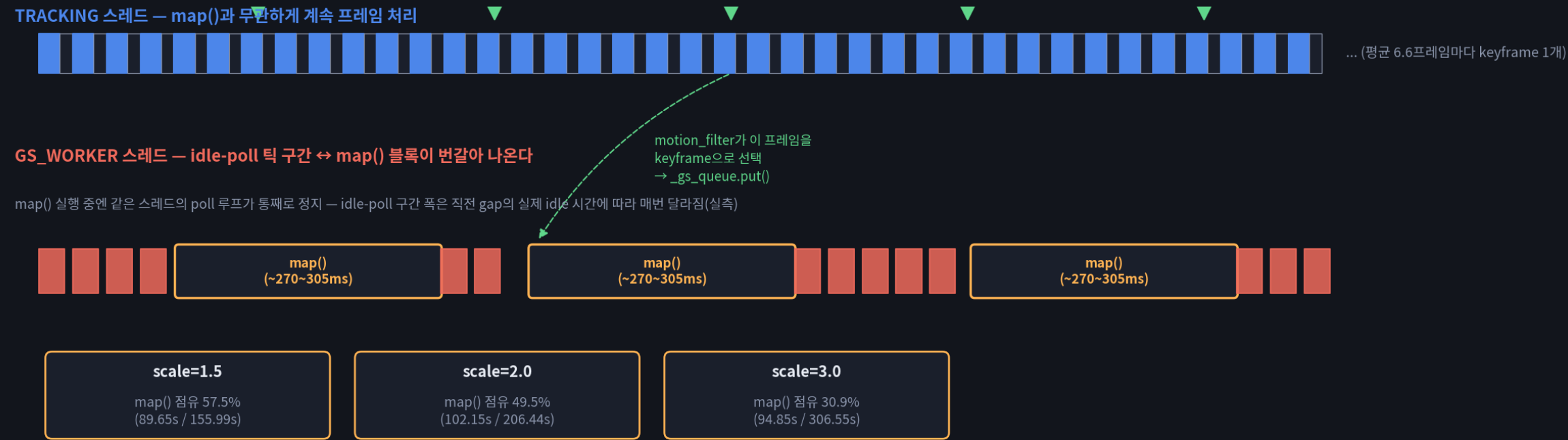


실제 총 횟수는 위 "프레임당 슬롯" 추정보다 훨씬 적다 — ① map() 실행 중엔 이 poll 루프 자체가 안 돌(같은 스레드) ② background_polish_start_frame 이전(전체 프레임의 53.7%)엔 애초에 비활성(map_mature_enough=False) — 다음 그림(매크로 타임라인)에서 설명

track(t)는 프레임마다 정확히 1번 — busy(추적 계산)와 idle(다음 프레임 예정시각까지 대기) 구간을 오가며, idle 구간에만 background_polish_step이 낚 수 있다.

map()이 주기적으로 폴리스 기회를 통째로 삼킨다

매크로 타임라인 — map()이 주기적으로 폴리스 기회를 통째로 삼킨다 (스케마틱, 실측 비율 반영)



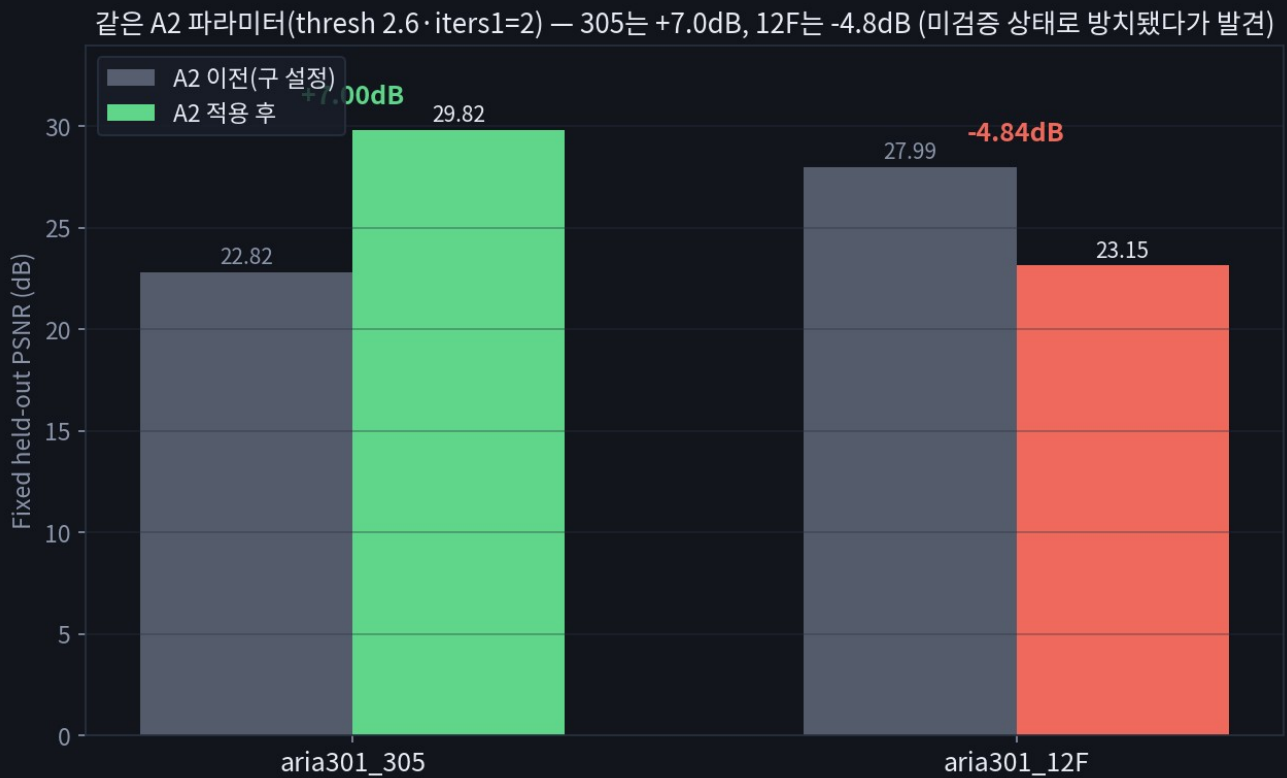
scale을 키우면 (1) 프레임당 idle 시간이 늘고 (2) map() 총 점유 비율도 같이 줄어든다 — 두 효과가 겹쳐 idle-poll 구간이 넓어지는 게 polish 급증(806→5,681)의 실제 원인.

GPU 실행 자체는 이 CPU 스레드 경계와 정확히 안 맞는다 — 실측: 세 스레드 총 busy 시간 합이 실제 벽시계 구간의 81~135%(scale별) → 부분적으로 겹쳐 실행됨(완전 직렬 아님, 공유 legacy default stream이라도).

polish 횟수를 결정하는 건 "프레임당 idle"과 "map()이 전체 시간의 몇 %를 먹는지" 두 가지 — scale을 올리면 이 둘이 같은 방향으로 움직여 폴리스 기회가 넓어진다.

EVIDENCE #1

같은 파라미터, 정반대 결과 — 실측



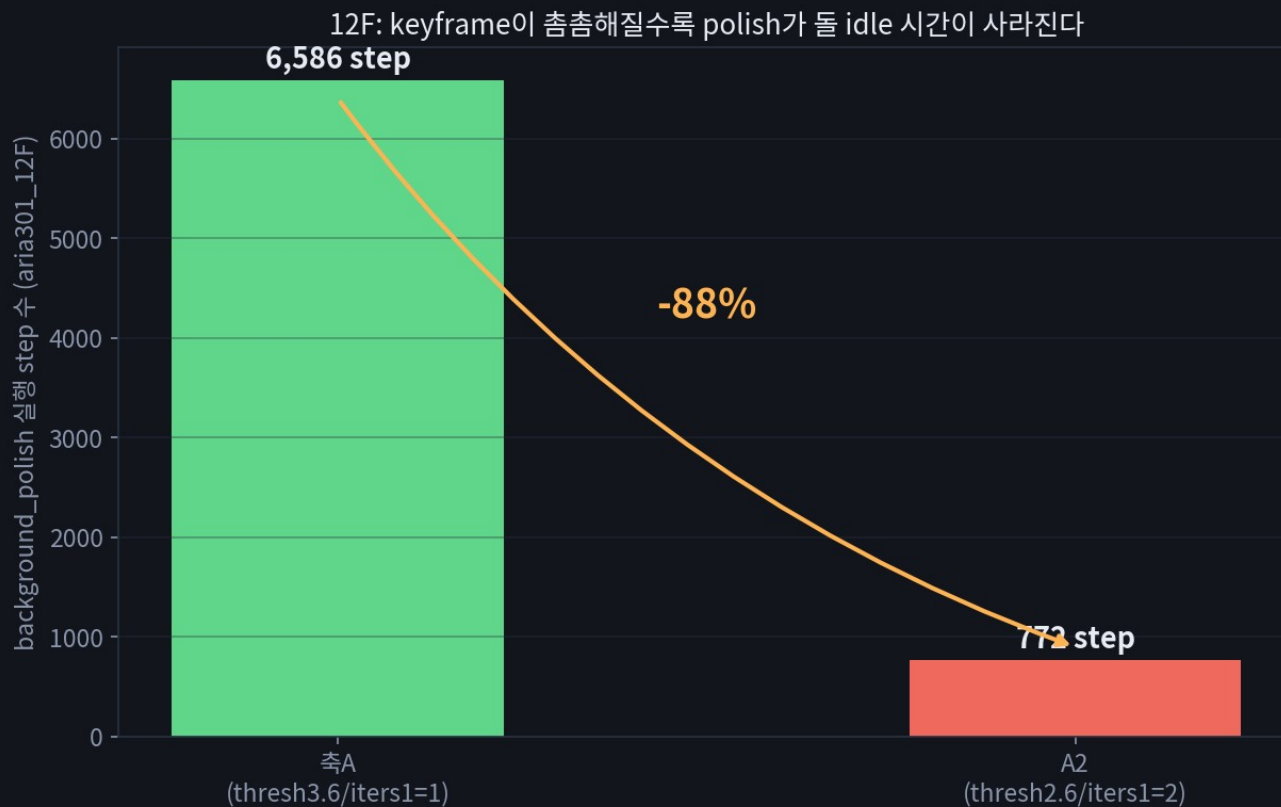
+7.00 dB
305 (원래 부족)

-4.84 dB
12F (원래 충분)

12F 미검증 방치
검증 상태

EVIDENCE #2

12F: 왜 떨어졌나 — background_polish가 줄었다



A2가 keyframe을 298→352개로 촘촘히 뽑으면서 tracking/mapping이 더 오래 바빠졌고, 그만큼 background_polish가 돌 idle 시간이 사라졌다(6,586→772 step, -88%). 12F는 원래(sparse 설정) 이미 keyframe 밀도가 충분한 장면(58.7개/프레임, 305는 29.7개/프레임)이라 — 촘촘하게 뽑는 "이득"은 거의 없이 "polish 손해"만 봤다.

같은 다이얼, 장면마다 반대로 작용하는 이유



※ 개념도 — 12F의 "이득 0"은 정밀 분해가 아니라 관측된 순효과(-4.84dB)를 설명하는 단순화된 예시다. 실제 이득/손해를 분리해서 재려면 아래 계측 정비가 선행돼야 한다. 핵심은 "keyframe 밀도"라는 하나의 다이얼이 두 개의 서로 다른 효과(커버리지 개선 vs polish 굽주림)를 동시에 만든다는 것.

있는 계측 vs 없는 계측 — 빈 구멍은 메웠다

VIGS_TIMING_LOG로 이미 커버됨 (이번 세션 내내 미사용)

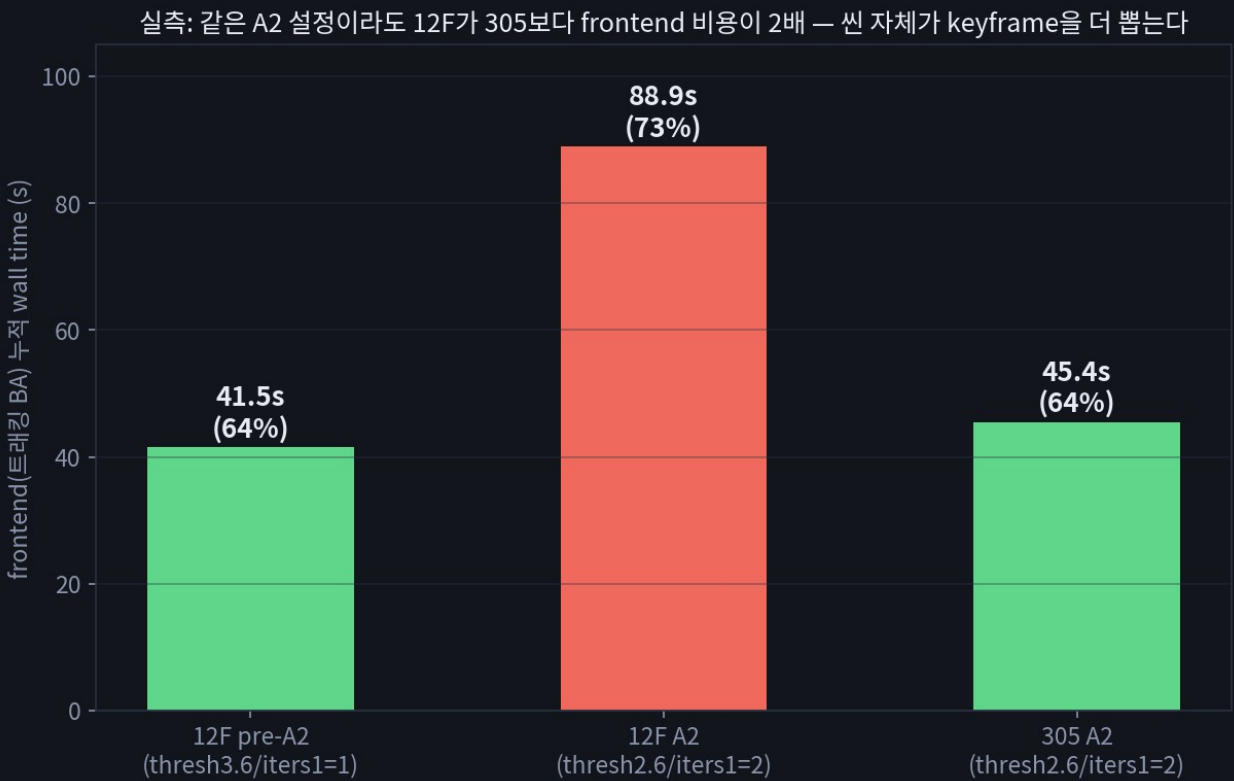
motion_filter	vigs.py:699
frontend (BA)	vigs.py:702
pgba_run / pgba_call_gs	vigs.py:720,732
map() 섹션별 (iters/n_view/n_gauss 포함)	gs_backend.py:3908

빠져있음 — 새로 계측 필요

background_polish_step 전체	gs_backend.py:1216-2121 — 계측 0건
---------------------------	---------------------------------

`VIGS\_TIMING\_LOG`는 이미 있었지만 이번 세션 내내 한 번도 안 켜진 상태였다 — 지금 켜서 4개 런(12F pre-A2/A2, 305 A2, 12F A2 재검증)을 재분석했다. 유일한 빈 구멍이었던 `background\_polish\_step` 내부에도 `\_Sect` 패턴으로 계측을 추가(`gs\_backend.py:1607,2099` 근처, `n_gauss/scope/batch_size` 컨텍스트 포함)해서 이제 완전히 커버된다.

실측 결과 ① — frontend가 진짜 지배적이다



73.4%
12F A2 frontend 비중

×2.17
iters1 1→2 배율

1.2~1.7%
gs_mapping(디스패치) 비중

실측 결과 ② — polish는 "느려진 게" 아니라 "기회가 없었다"

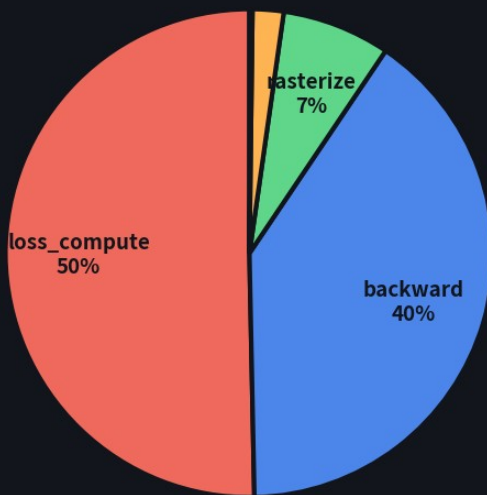


핵심 확인: call당 비용(3.9~6.4ms)은 gaussian 수(40k~150k)에 걸쳐 거의 고정 — 미세한 n_gauss 의존성은 있지만(약 1.5배 범위) 기회 횟수 차이(13배)에 비하면 훨씬 작다. 즉 exp카드에 남겼던 "772 step이 idle 부족 때문인지 step 자체가 느려서인지" 질문의 답은 확정적으로 idle 부족 쪽이다.

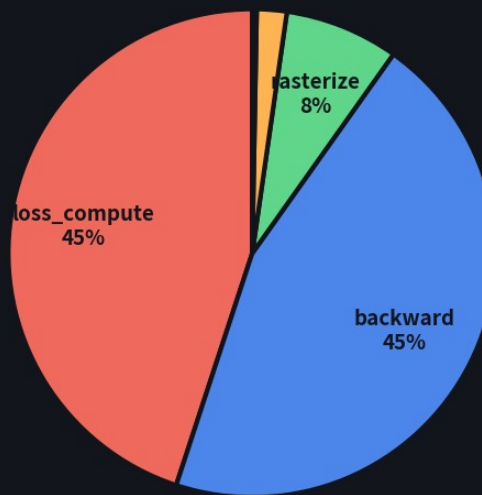
실측 결과 ③ — map() 자체는 두 씩 모두 같은 구성

map() 콜 내부 구성 — 두 씩 모두 loss_compute+backward가 ~90%, rasterize는 ~7%뿐

12F A2
map() 내부 75.2s

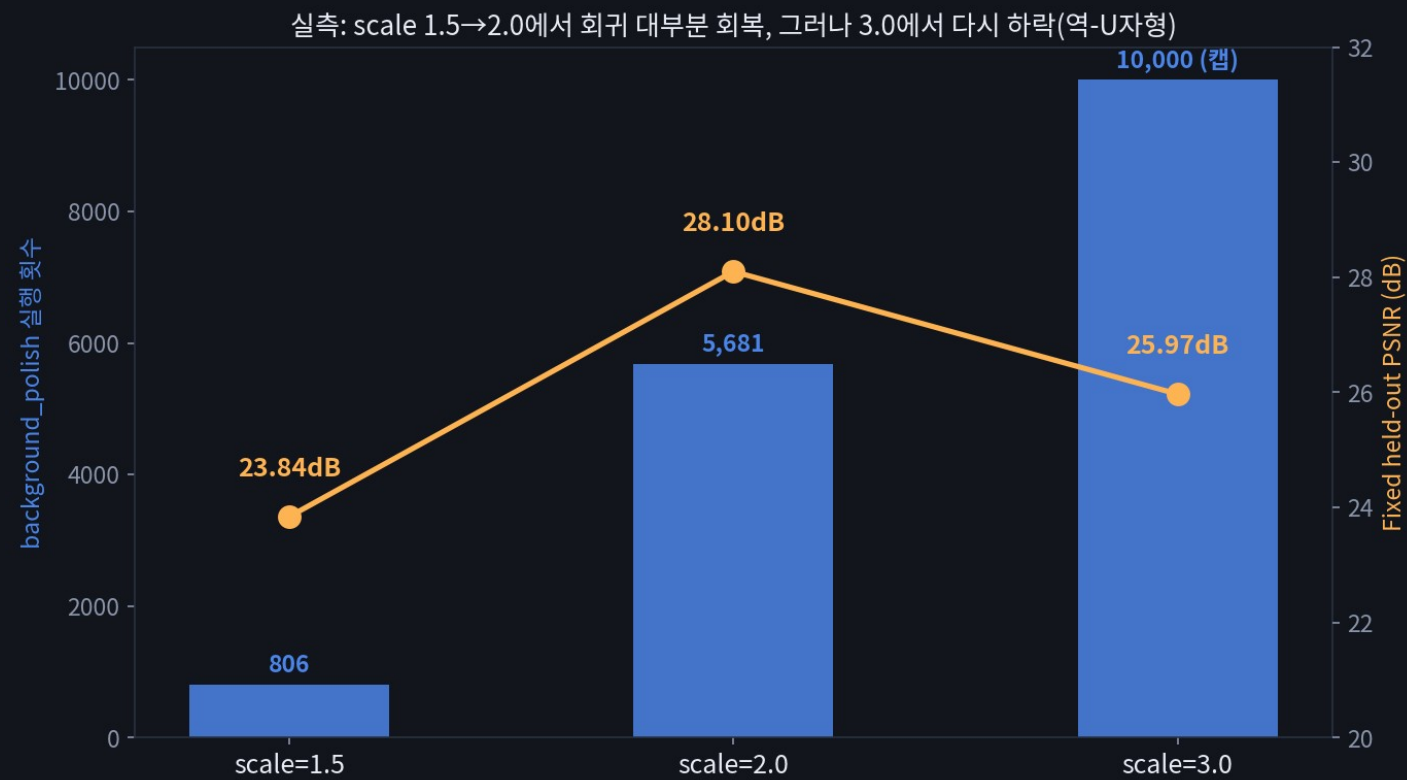


305 A2
map() 내부 41.3s



loss_compute+backward가 두 씩 모두 ~90%로 지배적, rasterize는 ~7~8%뿐 — map() 자체의 내부 구성은 씩에 따라 거의 안 변한다. 즉 12F가 나빠진 원인은 map() 내부가 아니라 map()이 dispatch되기 전 단계(frontend가 얼마나 자주·오래 도는가)에 있다는 게 다시 확인된다.

실측 결과 ④ — 예산을 33% 늘리면 회귀가 거의 회복된다 (역-U자형)



+4.26 dB
scale 1.5→2.0

-2.13 dB
scale 2.0→3.0

10,000 캡 도달
polish 3.0 상태

실측 결과 ⑤ — polish 횟수가 "계단식"으로 보였던 진짜 이유

keyframe 디스패치 사이 gap 구조 (실측, scale=1.5)



총 polish 횟수 = $\sum \text{floor}(\text{gap 길이} / \text{step비용 } 4\sim 6\text{ms})$

→ 정수 나눗셈의 합이라 본질적으로 "계단식(양자화)"

실측: gap의 43~57%는 step 1개(4~6ms)도 못 길 만큼 짧음(zero-polish gap).

총량은 소수의 긴 gap(최대 526→1582→2478ms, scale 1.5→2.0→3.0)이 지배.

gs_worker_dispatch/background_polish_call에 epoch 타임스탬프를 추가해 디스패치 사이 gap을 직접 재구성했다. 총 polish 횟수는 각 gap 길이를 step당 비용 (4~6ms)으로 나눈 정수 몫의 합 — gap의 43~57%는 아예 1개도 못 길 만큼 짧아 "부드러운 비례"가 아니라 본질적으로 계단식이다. scale이 커질수록 긴 gap(최대 526→1582→2478ms)이 늘어나며 총량이 늘지만, 3.0의 역행은 이 메커니즘만으로 설명 안 됨 — 좁은 후보 풀(stride-5 rgb_dense) 과적합 가설 필요(미검증).

map() freeze 시점 3지점 실측 — 두 극단 모두 기각

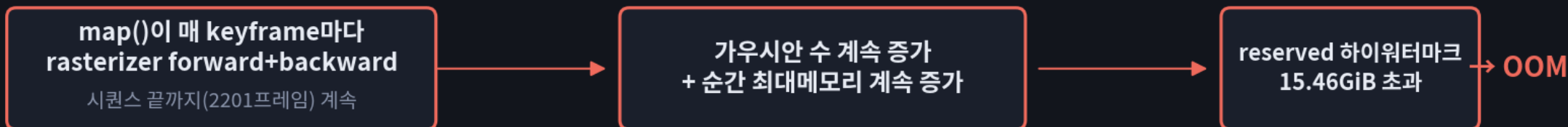


즉시 freeze(19.35dB)와 never freeze(24.90dB, 12F OOM) 둘 다 A2가 채택한 61% 지점(29.82dB)보다 나쁘다 — A2의 freeze 시점은 이미 근처 최적이며, 12F OOM은 "절대 freeze 안 함" 극단에서만 발생해 이미 기각된 조합이라 현재 채택 레시피엔 존재하지 않는다.

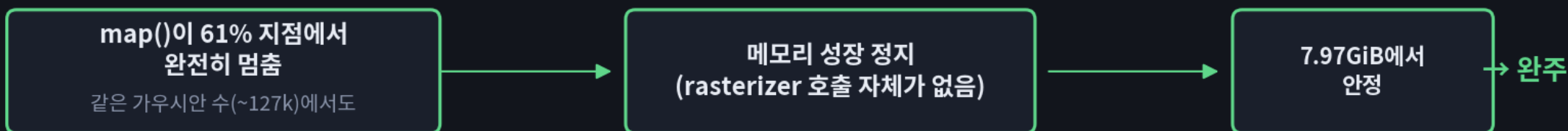
freeze가 실제로 막는 것 — 메모리 폭주, 그리고 예산 분배

"freeze"가 실제로 하는 일 — map()의 유일한 학습-비용 상한선

freeze 없음 (축 C)



freeze 있음 — A2 채택 지점 (61%)



핵심: 원인은 "가우시안 개수 자체"가 아니라 "freeze 없이 이 무거운 호출이 시퀀스 끝까지 반복된다"는 것 — 같은 가우시안 수(~127k)에서도 freeze 유무에 따라 7.97GiB ↔ 15.46GiB+ 로 갈렸다(12F 실측).

부가 효과: freeze 이후엔 map()이 GPU를 안 쓰니 background_polish가 그 시간을 넘겨받는다 — 즉 freeze 시점은 "메모리 안전판"이자 "구조생성↔정제 예산 분배선" 둘 다다.

FOUR DIRECTIONS

다음에 정해야 할 4가지 — 전부 같은 문제의 다른 얼굴

① FREEZE 기준

`mapping\_freeze\_after\_frac` 고정 비율(0.6140)은 1253/305 튜닝값일 뿐 원리적 근거 없음 — 12F 회귀가 직접 반례. coverage/densify-rate/loss-plateau 중 robust한 기준 필요.

② CARVE LOSS 이식

배치에서 검증된 방법론(exp44d2, 33.8dB)을 density/pruning 결정에 이식. 단, 12F 재튜닝 전이라 3장면 다 27dB 안정화 전 — CLAUDE.md 순서 제약과 맞물림.

③ 예산 robust화

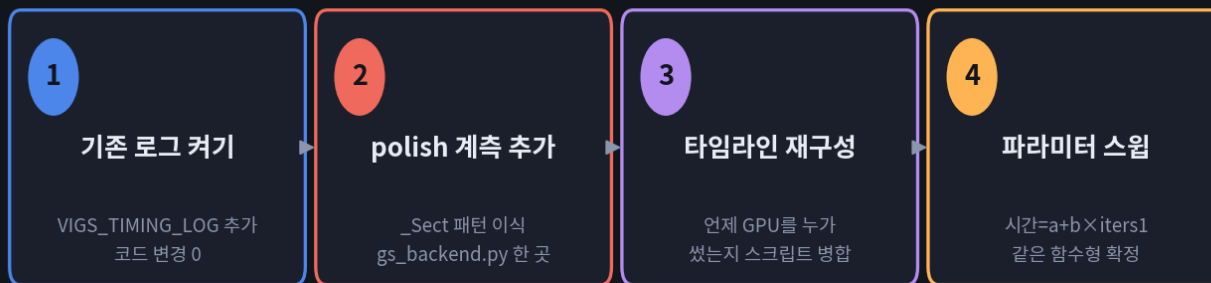
tracking에 시간 캡을 걸 것인가, polish에 최소 하한을 보장할 것인가 — 1253+305+12F 세 장면 교차검증 필수(A2가 두 장면만 보고 실패한 패턴 반복 금지).

④ FREEZE 스케줄

①과 본질적으로 같은 문제. iteration/frame count가 아닌 기준(coverage 신호 등)으로 대체 — 지금 죽어있는 `transmittance` 계산이 후보 재료.

①④(freeze 기준)와 ③(예산 배분)은 사실 하나의 문제 — "제한된 실시간 예산을 구조 생성과 정제에 어떻게 나누는가". ②(carve)는 그 정제 단계의 품질을 결정한다.

계획했던 4단계 중 어디까지 왔나

**STEP 1 완료**

A2 전/후 × 12F/305 4개 조합을 `VIGS\_TIMING\_LOG` 켜서 재실행·재분석 완료 — frontend 73.4%/64.0%, polish 5,683/822/9,866 회 등 실측 확보.

STEP 2 완료

`background\_polish\_step`에 `\_Sect` 계측 추가 완료 — call당 3.9~6.4ms, n_gauss 의존성 약함(주 원인 아님) 확정.

STEP 3 보류

시간축 병합 타임라인 재구성 — 이번엔 phase별 총합/스케일링으로 핵심 질문에 이미 답이 나와서 우선순위 낮춤. `get\_lock()` 경합의 직접 증거는 아직 없음.

STEP 4 완료

파라미터 스왑 — iters1 1↔2 한 쌍(×2.17) + `replay\_time\_scale` 1.5/2.0/3.0 3점 스왑(23.84/28.10/25.97dB, 역-U자형) 확보. gap 기반 양자화 메커니즘도 규명 완료. 3.0 역행의 근본원인(과적합 가설)만 미검증으로 남음.

CONCLUSIONS

이번 조사로 확정된 것 · 다음 단계

확정된 것 (실측 완료)

① frontend가 프레임 단계 시간의 64~73%로 압도적 지배 — 12F는 305보다 절대시간 2배. ② background_polish는 "느려진 게" 아니라 "기회가 없었다"(call당 비용 거의 고정, 기회 횟수만 13배 차이) — exp카드 미해결 질문 해소. ③ map() 내부 구성(loss/backward ~90%)은 두 씩 모두 동일 — 문제는 map() 안이 아니라 그 앞 단계에 있다. ④ 예산을 33% 늘리면(scale 1.5→2.0) 12F 회귀가 거의 회복된다(+4.26dB) — 예산 부족이 인과관계임을 직접 실험으로 확인. ⑤ polish 횟수의 "계단식" 양상은 $\text{gap} \div \text{step}$ 비용의 정수 몫 합이라는 구조적 필연 — 신비가 아니라 산수.

아직 안 밝혀진 것

iters1 1→2가 왜 정확히 2.17배인지(keyframe 수 증가와 순수 iters 효과가 섞여있어 분리 안 됨) — 3장면 동시 통제 스위치 필요. `get_lock()` GPU 경합이 실측 병목인지도 직접 증거는 아직 없음(현재는 순차 phase 합산 기준). scale 3.0의 역행(28.10→25.97dB, polish는 오히려 최대인데도 하락)은 좁은 후보 풀 과적합 가설뿐 — held-out 분리 재검증 필요.

4방향에 대한 함의

①④(freeze 기준)와 ③(예산 robust화) 설계는 이제 "frontend를 줄이거나 polish에 최소 몫을 직접 보장" 둘 중 하나로 좁혀졌다 — 감이 아니라 실측 근거로. scale 스위치는 ③이 실제로 유효한 레버임을 증명했지만, 무작정 늘리는 게 답이 아니라(3.0 역행) "적정 하한 보장" 설계가 필요함도 같이 보여줬다. ②(carve)는 여전히 12F 재튜닝 이후로 순서 유지.